

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN – KHOA KHOA HỌC DỮ LIỆU & TRÍ
TUỆ NHÂN TẠO

DSAI1002: Cấu trúc Dữ liệu và Giải thuật với Python

BÀI GIẢNG CHUYÊN ĐỀ: PHẦN 03
Thuật toán và Phân tích Độ phức tạp

Giảng viên: TS. Vũ Đức Minh | Học kỳ: Mùa Thu 2026

Tài liệu học tập toàn diện về Thiết kế Thuật toán, Ký hiệu Tiệm cận, Phân tích Vòng lặp, Hệ thức Truy hồi và Khấu hao

Khung Kiến thức Cốt lõi của Phần 03

Tài liệu này hệ thống hóa toàn bộ nội dung lý thuyết, phân tích toán học và bài tập của **Phần 03: Thuật toán và Phân tích Độ phức tạp**:

- Khái niệm Thuật toán & Quy trình Thiết kế:** Định nghĩa chuẩn mực, phân biệt thuật toán và chương trình, các đặc tính cốt lõi (tính dừng, xác định, khả thi), lưu đồ, mã giả, quy trình 8 bước thiết kế giải thuật và so sánh phân tích tiên nghiệm (a priori) với hậu nghiệm (a posteriori).
- Ký hiệu Tiệm cận ($O, \Omega, \Theta, o, \omega$):** Bản chất toán học của phân tích tiệm cận khi $n \rightarrow \infty$, định nghĩa hình thức, giới hạn L'Hôpital, phân cấp các hàm tăng trưởng và phân biệt độ phức tạp bộ nhớ với bộ nhớ phụ trợ (auxiliary space).
- Phân tích Mã nguồn & Vòng lặp Không Độ quy:** Phương pháp tính toán chi tiết cho vòng lặp tuyến tính, lôgarit, vòng lặp tam giác phụ thuộc chỉ số, chuỗi hình học và chuỗi điều hòa (harmonic sum).
- Hệ thức Truy hồi & Định lý Thợ (Master Theorem):** Xây dựng phương trình đệ quy chia để trị $T(n) = aT(n/b) + f(n)$, phân tích cây đệ quy, định lý Master chuẩn (3 trường hợp), định lý Master mở rộng ($\Theta(n^k \log^p n)$), các trường hợp vi phạm và phương pháp đoán nghiệm quy nạp.
- Phân tích Khấu hao (Amortized Analysis):** Bản chất chi phí chuỗi tồi nhất, phương pháp cộng dồn (aggregate), phương pháp kế toán (accounting) và phương pháp hàm thế (potential) qua mô hình mảng động mở rộng gấp đôi (dynamic array doubling).

Mục lục

1	Thuật toán là gì? Nhập môn Tư duy Giải thuật	2
1.1	Ý niệm Trực giác và Định nghĩa Hình thức	2
1.2	Phân biệt Thuật toán và Chương trình Máy tính	2
1.3	Sáu Đặc tính Thiết yếu của một Thuật toán Chuẩn mực	2
1.4	Ba Phương pháp Biểu diễn Thuật toán Phổ biến	3
1.5	Quy trình Kỹ thuật 8 Bước Thiết kế Thuật toán	3
1.6	Phân tích Tiên nghiệm (A Priori) và Hậu nghiệm (A Posteriori)	4
2	Độ phức tạp Thuật toán và Ký hiệu Tiệm cận	5
2.1	Bản chất của Phân tích Tiệm cận ($n \rightarrow \infty$)	5
2.2	Ba Ký hiệu Tiệm cận Căn bản	5
2.3	Phương pháp Giới hạn Phân loại Cấp tăng trưởng	5
2.4	Bảng Phân cấp Tăng trưởng Chuẩn mực	6

2.5	Quy tắc Tính toán Độ phức tạp Thời gian	6
2.6	Độ phức tạp Bộ nhớ và Bộ nhớ Phụ trợ	6
3	Phân tích Mã nguồn & Vòng lặp Không Đệ quy	7
3.1	Vòng lặp Tuyến tính: $\Theta(n)$	7
3.2	Vòng lặp Lôgarit (Nhân / Chia): $\Theta(\log n)$	7
3.3	Vòng lặp Lồng nhau Dạng Tam giác: $\Theta(n^2)$	7
3.4	Vòng lặp Thu hẹp Cấp số nhân: $\Theta(n)$	7
3.5	Vòng lặp Chuỗi Điều hòa (Harmonic Sum): $\Theta(n \log n)$	7
3.6	Điều kiện Lặp Phi Tuyến: $\Theta(\sqrt{n})$	8
4	Hệ thức Truy hồi & Định lý Thợ (Master Theorem)	9
4.1	Phương trình Truy hồi Chia để trị	9
4.2	Mô hình Cây Đệ quy (Recursion Tree)	9
4.3	Định lý Master Chuẩn mực (CLRS)	9
4.4	Định lý Master Mở rộng (Extended Master Theorem)	9
4.5	Truy hồi Dạng Subtract-and-Conquer	10
5	Phân tích Khấu hao (Amortized Analysis)	11
5.1	Ý nghĩa và Bản chất	11
5.2	Ba Phương pháp Phân tích Khấu hao	11
5.3	Nghiên cứu Điển hình: Cơ chế Gấp đôi Dung lượng của Mảng Động	11
6	Bài tập Thực hành & Lời giải Chi tiết	13
6.1	Bài toán 1: Phân cấp Tốc độ Tăng trưởng Tiệm cận	13
6.2	Bài toán 2: Phân tích Vòng lặp Lồng nhau	13
6.3	Bài toán 3: Ứng dụng Định lý Master	13
7	Bảng Tra cứu Công thức & Tóm tắt Nhanh	15
7.1	Các Công thức Tổng Cốt lõi	15
7.2	Bảng Quyết định Định lý Master	15

1. Thuật toán là gì? Nhập môn Tư duy Giải thuật

1.1 Ý niệm Trực giác và Định nghĩa Hình thức

Trong đời sống thường ngày, chúng ta liên tục áp dụng tư duy giải thuật: nấu một món ăn theo công thức, tra cứu một số điện thoại, lựa chọn lộ trình ngắn nhất trên bản đồ hay phân phối đơn hàng cho xe giao vận. Khi các chỉ dẫn này được chuẩn hóa với tính chính xác tuyệt đối, hữu hạn bước và có thể thi hành bởi máy tính, chúng trở thành **thuật toán**.

Định nghĩa 1.1 (Thuật toán – Algorithm)

Một **thuật toán** là một dãy hữu hạn các chỉ dẫn rõ ràng, không mâu thuẫn, được xác định chuẩn xác nhằm giải quyết một lớp bài toán tính toán cụ thể, hoàn thành một tác vụ tự động hoặc tính giá trị của một hàm số. Thuật toán nhận dữ liệu đầu vào (Input), trải qua các bước biến đổi trạng thái xác định hoặc ngẫu nhiên, và tạo ra kết quả đầu ra (Output) sau một số hữu hạn bước tính toán.

Mô hình trừu tượng của quá trình xử lý thuật toán gồm 3 khối kinh điển:



1.2 Phân biệt Thuật toán và Chương trình Máy tính

Một sai lầm phổ biến ở người học là đồng nhất *thuật toán* với *chương trình*. Thuật toán đại diện cho ý tưởng logic và giải pháp toán học trừu tượng, trong khi chương trình là hiện thực cụ thể của thuật toán bằng một ngôn ngữ lập trình xác định.

Tiêu chí	Thuật toán (Algorithm)	Chương trình (Program)
Bản chất	Ý tưởng giải quyết vấn đề, quy trình logic	Bản hiện thực cụ thể có thể biên dịch/thực thi
Ngôn ngữ	Độc lập với ngôn ngữ (mã giả, ngôn ngữ tự nhiên)	Python, C++, Rust, Java, Go, Assembly
Mức độ chi tiết	Tập trung vào luồng logic và tính bất biến	Đầy đủ cú pháp, kiểu dữ liệu, thư viện, quản lý lỗi
Đối tượng tiếp nhận	Con người (nhà nghiên cứu, kỹ sư lập trình)	Trình biên dịch/thông dịch và phần cứng CPU
Tuổi thọ	Bền vững hàng thế kỷ (ví dụ thuật toán Euclid: 300 TCN)	Phụ thuộc vào vòng đời ngôn ngữ và phiên bản thư viện

1.3 Sáu Đặc tính Thiết yếu của một Thuật toán Chuẩn mực

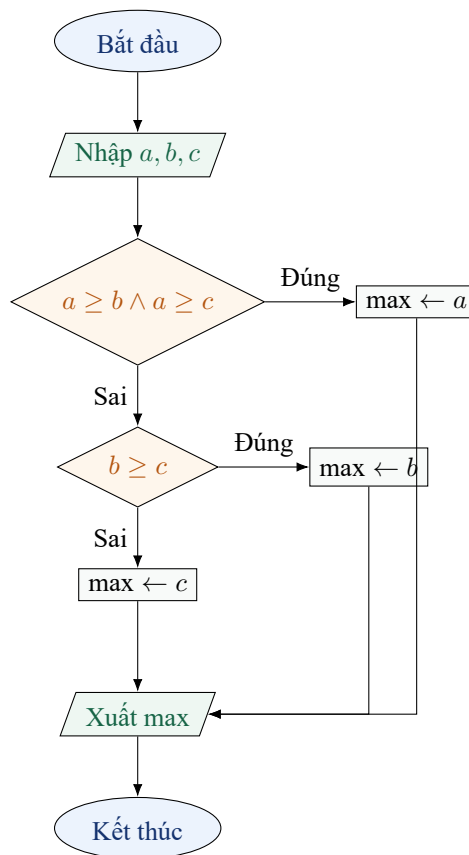
Theo Donald Knuth (*The Art of Computer Programming*), mọi thuật toán hợp lệ đều phải đáp ứng đầy đủ 6 đặc tính sau:

- Tính Xác định (Definiteness / Unambiguity):** Mỗi bước chỉ dẫn phải có một và chỉ một cách hiểu duy nhất. Các mô tả mập mờ như “chọn một số đủ lớn” hoặc “sắp xếp nhanh danh sách” đều không hợp lệ.
- Đầu vào Rõ ràng (Defined Input):** Thuật toán nhận không hoặc nhiều đầu vào; các đầu vào phải được chỉ định rõ kiểu dữ liệu, miền giá trị và điều kiện biên.
- Đầu ra Xác định (Defined Output):** Thuật toán phải tạo ra ít nhất một kết quả đầu ra có quan hệ toán học được xác định với đầu vào.

4. **Tính Dừng (Finiteness):** Thuật toán bắt buộc phải kết thúc sau một số bước hữu hạn đối với mọi trường hợp đầu vào hợp lệ. Vòng lặp vô tận không bao giờ dừng không thể gọi là thuật toán.
5. **Tính Khả thi (Effectiveness / Feasibility):** Mỗi thao tác phải đủ sơ cấp để có thể thực thi chính xác bởi máy tính hoặc con người trên giấy bút với lượng tài nguyên hữu hạn.
6. **Tính Đúng đắn (Correctness):** Thuật toán phải sinh ra đầu ra chính xác như đặc tả bài toán cho tất cả các trường hợp đầu vào hợp lệ.

1.4 Ba Phương pháp Biểu diễn Thuật toán Phổ biến

1. **Ngôn ngữ Tự nhiên:** Thân thiện, dễ hiểu cho người mới bắt đầu nhưng dễ mơ hồ và thiếu cấu trúc khi mô tả logic phức tạp.
2. **Lưu đồ (Flowchart):** Biểu diễn trực quan bằng sơ đồ hình học chuẩn ANSI (Hình oval cho Bắt đầu/Kết thúc; Hình bình hành cho Vào/Ra; Hình chữ nhật cho Xử lý; Hình thoi cho Phân nhánh điều kiện).
3. **Mã giả (Pseudocode):** Ngôn ngữ bán hình thức mô tả các cấu trúc điều khiển (IF, ELSE, WHILE, FOR) mà không bị phụ thuộc vào cú pháp rườm rà của bất kỳ ngôn ngữ cụ thể nào.



1.5 Quy trình Kỹ thuật 8 Bước Thiết kế Thuật toán

1. **Bước 1: Xác định Bài toán.** Nắm rõ mục tiêu cần giải quyết.
2. **Bước 2: Xác định Đầu vào.** Kiểu dữ liệu, kích thước dữ liệu n , các giả thiết tiên quyết.
3. **Bước 3: Xác định Đầu ra.** Giá trị trả về, cấu trúc dữ liệu kết quả, trạng thái thay đổi.
4. **Bước 4: Nhận diện Ràng buộc.** Giới hạn thời gian (ví dụ 1 giây $\implies \approx 10^8$ thao tác), giới hạn bộ nhớ phụ trợ ($O(1)$ hay $O(n)$).

5. **Bước 5: Đề xuất Ý tưởng.** Lựa chọn chiến lược (Chia để trị, Quy hoạch động, Tham lam, Duyệt toàn bộ).
6. **Bước 6: Viết Mã giả / Lưu đồ.** Làm rõ cấu trúc lặp, điều kiện dừng và bất biến lặp.
7. **Bước 7: Kiểm thử Ca đặc biệt.** Kiểm tra danh sách rỗng, mảng 1 phần tử, phần tử trùng lặp, số âm.
8. **Bước 8: Phân tích Độ phức tạp.** Ước lượng chi phí thời gian và bộ nhớ phụ trợ bằng ký hiệu tiệm cận.

1.6 Phân tích Tiên nghiệm (A Priori) và Hậu nghiệm (A Posteriori)

Tiêu chuẩn	Phân tích Tiên nghiệm (A Priori)	Phân tích Hậu nghiệm (A Posteriori)
Thời điểm	Trước khi viết mã trên máy tính	Sau khi cài đặt và chạy trên máy tính
Phương pháp	Mô hình toán học đếm phép toán cơ bản	Đo thời gian chạy thực tế ('time'), bộ nhớ tiêu thụ ('tracemalloc')
Phụ thuộc phần cứng	Không phụ thuộc (Khách quan với mọi máy)	Phụ thuộc rất lớn (CPU, RAM, Cache, Hệ điều hành)
Mục tiêu	Xác định quy luật tăng trưởng khi $n \rightarrow \infty$	Đánh giá hiệu năng thực tế trên một tập dữ liệu cụ thể
Biểu diễn	Ký hiệu toán học Big- O , Big- Ω , Big- Θ	Bảng số liệu giây/mili-giây, đồ thị thực nghiệm

2. Độ phức tạp Thuật toán và Ký hiệu Tiệm cận

2.1 Bản chất của Phân tích Tiệm cận ($n \rightarrow \infty$)

Phân tích tiệm cận tập trung vào cách lượng tài nguyên (thời gian $T(n)$ hoặc bộ nhớ $S(n)$) tăng lên khi quy mô dữ liệu n trở nên vô cùng lớn ($n \rightarrow \infty$). Hai nguyên lý tối giản căn bản:

1. **Bỏ qua hằng số nhân:** Thuật toán chạy $2n$ hay $100n$ bước đều có cùng tốc độ tăng trưởng tuyến tính.
2. **Bỏ qua số hạng bậc thấp:** Với $f(n) = 3n^2 + 500n + 10000$, khi $n = 10^6$, số hạng $3n^2 = 3 \times 10^{12}$ lấn át hoàn toàn $500n = 5 \times 10^8$.

2.2 Ba Ký hiệu Tiệm cận Căn bản

Định nghĩa 2.1 (Ký hiệu Big O – Chặn trên Tiệm cận)

Cho $f(n)$ và $g(n)$ là các hàm số không âm. Ta nói $f(n) = O(g(n))$ nếu tồn tại các hằng số dương $c > 0$ và $n_0 \geq 1$ sao cho:

$$0 \leq f(n) \leq c \cdot g(n) \quad \text{với mọi } n \geq n_0.$$

Ý nghĩa: $g(n)$ là mức chặn trên về tốc độ tăng trưởng của thuật toán. Trong **trường hợp xấu nhất**, thời gian chạy sẽ không bao giờ vượt quá tốc độ của $g(n)$.

Định nghĩa 2.2 (Ký hiệu Big Ω – Chặn dưới Tiệm cận)

Ta nói $f(n) = \Omega(g(n))$ nếu tồn tại các hằng số dương $c > 0$ và $n_0 \geq 1$ sao cho:

$$0 \leq c \cdot g(n) \leq f(n) \quad \text{với mọi } n \geq n_0.$$

Ý nghĩa: $g(n)$ là mức chặn dưới. Trong mọi trường hợp, thuật toán phải tốn **ít nhất** chừng đó phép tính khi n đủ lớn.

Định nghĩa 2.3 (Ký hiệu Big Θ – Chặn chặt Tiệm cận)

Ta nói $f(n) = \Theta(g(n))$ nếu tồn tại các hằng số $c_1, c_2 > 0$ và $n_0 \geq 1$ sao cho:

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{với mọi } n \geq n_0.$$

Định lý tương đương: $f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ và } f(n) = \Omega(g(n)).$

2.3 Phương pháp Giới hạn Phân loại Cấp tăng trưởng

Thay vì tìm trực tiếp các hằng số c và n_0 , ta khảo sát giới hạn tỉ số khi $n \rightarrow \infty$:

$$L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

- Nếu $L = 0$: $f(n) = o(g(n))$ (và do đó $f(n) = O(g(n))$, nhưng $f(n) \neq \Theta(g(n))$). Hàm f tăng chậm hơn ngất so với g .
- Nếu $0 < L < \infty$: $f(n) = \Theta(g(n))$. Hai hàm có cùng bậc tăng trưởng tiệm cận.
- Nếu $L = \infty$: $f(n) = \omega(g(n))$ (và $f(n) = \Omega(g(n))$). Hàm f tăng nhanh hơn ngất so với g .

2.4 Bảng Phân cấp Tăng trưởng Chuẩn mực

Thứ tự phân cấp từ chậm nhất đến nhanh nhất:

$O(1) \prec O(\log \log n) \prec O(\log n) \prec O(\sqrt{n}) \prec O(n) \prec O(n \log n) \prec O(n^2) \prec O(n^3) \prec O(2^n) \prec O(n!) \prec O(n^n)$

Ký hiệu	Tên gọi	Ví dụ Giải thuật Điển hình
$O(1)$	Hằng số (Constant)	Truy cập phần tử mảng 'arr[i]', tra cứu Hash Table (trung bình)
$O(\log n)$	Lôgarit (Logarithmic)	Tìm kiếm nhị phân (Binary Search), thao tác trên cây BST cân bằng
$O(\sqrt{n})$	Căn bậc hai (Sub-linear)	Kiểm tra số nguyên tố bằng thử ước, thuật toán Mo
$O(n)$	Tuyến tính (Linear)	Tìm kiếm tuyến tính, duyệt mảng 1 lần, tìm Max/Min
$O(n \log n)$	Tuyến tính-lôgarit	Sắp xếp tối ưu (Merge Sort, Heap Sort, Quick Sort trung bình)
$O(n^2)$	Bậc hai (Quadratic)	Sắp xếp nổi bọt (Bubble Sort), chọn (Selection), chèn (Insertion)
$O(n^3)$	Bậc ba (Cubic)	Nhân hai ma trận vuông kích thước $n \times n$ theo định nghĩa, Floyd-Warshall
$O(2^n)$	Hàm mũ (Exponential)	Liệt kê tất cả tập con (Power Set), Fibonacci đệ quy ngây thơ
$O(n!)$	Giai thừa (Factorial)	Sinh mọi hoán vị của n phần tử, Người du lịch duyệt vét cạn

2.5 Quy tắc Tính toán Độ phức tạp Thời gian

1. **Quy tắc Cộng (Khởi lệnh Tuần tự):** Nếu hai đoạn mã thực thi nối tiếp có chi phí $O(f(n))$ và $O(g(n))$, tổng chi phí là:

$$T(n) = O(f(n) + g(n)) = O(\max(f(n), g(n)))$$

2. **Quy tắc Nhân (Vòng lặp Lồng nhau):** Nếu vòng lặp ngoài chạy $f(n)$ lần và tại mỗi bước vòng lặp trong thực hiện $g(n)$ phép tính:

$$T(n) = O(f(n) \cdot g(n))$$

2.6 Độ phức tạp Bộ nhớ và Bộ nhớ Phụ trợ

- Tổng Độ phức tạp Bộ nhớ (Space Complexity):** Toàn bộ lượng RAM thuật toán chiếm dụng, bao gồm cả không gian lưu trữ dữ liệu đầu vào.
- Bộ nhớ Phụ trợ (Auxiliary Space):** Bộ nhớ *bổ sung tạm thời* do chính thuật toán cấp phát thêm trong quá trình tính toán (biến phụ, mảng tạm, và các khung ngăn xếp gọi hàm đệ quy).

3. Phân tích Mã nguồn & Vòng lặp Không Độ quy

3.1 Vòng lặp Tuyến tính: $\Theta(n)$

```
1 i = 0
2 while i < n:
3     # Các thao tác  $O(1)$ 
4     i += c # c là hằng số dương
```

Số bước lặp là $\lceil n/c \rceil$. Khi bỏ hằng số $1/c$, ta có độ phức tạp $\Theta(n)$.

3.2 Vòng lặp Lôgarit (Nhân / Chia): $\Theta(\log n)$

```
1 i = 1
2 while i < n:
3     # Thao tác  $O(1)$ 
4     i *= 2
```

Sau k bước lặp, giá trị $i = 2^k$. Vòng lặp dừng khi $2^k \geq n \iff k = \lceil \log_2 n \rceil$. Độ phức tạp là $\Theta(\log n)$.

3.3 Vòng lặp Lồng nhau Dạng Tam giác: $\Theta(n^2)$

```
1 for i in range(n):
2     for j in range(i, n):
3         # Thao tác  $O(1)$ 
4         process(i, j)
```

Tổng số phép tính:

$$S = \sum_{i=0}^{n-1} (n-i) = n + (n-1) + \dots + 1 = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n \implies \Theta(n^2)$$

3.4 Vòng lặp Thu hẹp Cấp số nhân: $\Theta(n)$

```
1 def geometric_loop(n):
2     i = n
3     while i > 0:
4         for j in range(i):
5             print(j)
6         i //= 2
```

Biến i nhận các giá trị $n, n/2, n/4, \dots, 1$. Tổng số bước lặp được chặn bởi chuỗi hình học vô hạn:

$$S(n) = n + \frac{n}{2} + \frac{n}{4} + \dots \leq n \sum_{k=0}^{\infty} \left(\frac{1}{2}\right)^k = 2n = \Theta(n)$$

Lưu ý: Mặc dù vòng lặp ngoài chạy $O(\log n)$ lần và vòng trong chạy tối đa n lần, kết quả chính xác là $\Theta(n)$, không phải $O(n \log n)$.

3.5 Vòng lặp Chuỗi Điều hòa (Harmonic Sum): $\Theta(n \log n)$

```
1 for i in range(1, n + 1):
2     j = i
3     while j <= n:
4         j += i
```

Tổng số phép toán là:

$$T(n) = \sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor \approx n \left(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \right) = n \cdot H_n = \Theta(n \log n)$$

3.6 Điều kiện Lặp Phi Tuyến: $\Theta(\sqrt{n})$

```
1 i = 1
2 while i * i <= n:
3     i += 1
```

Điều kiện $i^2 \leq n \iff i \leq \sqrt{n}$. Số bước lặp là $\lfloor \sqrt{n} \rfloor \implies \Theta(\sqrt{n})$.

4. Hệ thức Truy hồi & Định lý Thợ (Master Theorem)

4.1 Phương trình Truy hồi Chia để trị

Một bài toán kích thước n được chia thành a bài toán con kích thước n/b , giải đệ quy và tổng hợp nghiệm với chi phí $f(n)$:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

với $a \geq 1$, $b > 1$ và $f(n)$ là hàm không âm khi n đủ lớn.

4.2 Mô hình Cây Đệ quy (Recursion Tree)

- Chiều cao cây:** Kích thước giảm dần $n/b^j = 1 \implies$ Độ sâu $D = \log_b n$.
- Số nút tại tầng j :** a^j nút, mỗi nút tốn công sức $f(n/b^j)$.
- Số nút lá tại đáy cây:** $a^{\log_b n} = n^{\log_b a}$ lá.
- Tổng chi phí tầng lá:** $\Theta(n^{\log_b a})$.

4.3 Định lý Master Chuẩn mực (CLRS)

Định lý 4.1 (Master Theorem cho Đệ quy Chia để trị)

Xét phương trình $T(n) = aT(n/b) + f(n)$. Hành vi tiệm cận được xác định bằng việc so sánh $f(n)$ với đại lượng chuẩn $n^{\log_b a}$:

- Trường hợp 1 (Phần đệ quy chi phối):** Nếu tồn tại $\epsilon > 0$ sao cho $f(n) = O(n^{\log_b a - \epsilon})$, thì:

$$T(n) = \Theta(n^{\log_b a})$$

- Trường hợp 2 (Cân bằng giữa đệ quy và chia/gộp):** Nếu $f(n) = \Theta(n^{\log_b a})$, thì:

$$T(n) = \Theta(n^{\log_b a} \log n)$$

- Trường hợp 3 (Phần ngoài đệ quy chi phối):** Nếu tồn tại $\epsilon > 0$ sao cho $f(n) = \Omega(n^{\log_b a + \epsilon})$, đồng thời thỏa mãn điều kiện chính quy:

$$a \cdot f(n/b) \leq c \cdot f(n) \quad \text{với hằng số } c < 1 \text{ và mọi } n \text{ đủ lớn,}$$

thì:

$$T(n) = \Theta(f(n))$$

4.4 Định lý Master Mở rộng (Extended Master Theorem)

Áp dụng cho dạng tổng quát $T(n) = aT(n/b) + \Theta(n^k \log^p n)$:

Điều kiện	Số mũ Lôgarit p	Nghiệm $T(n)$	
$a > b^k$	Mọi $p \in \mathbb{R}$	$\Theta(n^{\log_b a})$	(Đệ quy chiếm ưu thế tuyệt đối)
$a = b^k$	$p > -1$	$\Theta(n^k \log^{p+1} n)$	(Dạng mở rộng cơ bản)
	$p = -1$	$\Theta(n^k \log \log n)$	(Trường hợp biên cực hạn)
	$p < -1$	$\Theta(n^k)$	(Tổng chuỗi hội tụ)
$a < b^k$	$p \geq 0$	$\Theta(n^k \log^p n)$	(Chi phí tại gốc chiếm ưu thế)
	$p < 0$	$O(n^k)$	

4.5 Truy hồi Dạng Subtract-and-Conquer

Khi kích thước bài toán giảm đi một lượng trừ cố định $b > 0$:

$$T(n) = aT(n - b) + f(n), \quad \text{với } f(n) = O(n^k)$$

Điều kiện của a	Độ phức tạp $T(n)$	Ví dụ Minh họa
$a < 1$	$O(f(n)) = O(n^k)$	$T(n) = 0.5T(n - 1) + n^2 \implies O(n^2)$
$a = 1$	$O(n \cdot f(n)) = O(n^{k+1})$	$T(n) = T(n - 1) + n \implies O(n^2)$ (Selection Sort)
$a > 1$	$O(f(n) \cdot a^{n/b}) = O(n^k a^{n/b})$	$T(n) = 2T(n - 1) + 1 \implies O(2^n)$ (Tháp Hà Nội)

5. Phân tích Khấu hao (Amortized Analysis)

5.1 Ý nghĩa và Bản chất

Trong thực tế lập trình, có những cấu trúc dữ liệu mà đa số các thao tác diễn ra vô cùng nhanh ($O(1)$), nhưng thỉnh thoảng xuất hiện một thao tác rất tốn kém ($O(n)$) nhằm tái cấu trúc bộ nhớ. Phân tích trường hợp xấu nhất cho từng thao tác riêng lẻ sẽ dẫn đến kết luận bi quan quá mức.

Định nghĩa 5.1 (Chi phí Khấu hao)

Trong một chuỗi gồm m thao tác liên tiếp, nếu tổng thời gian chạy thực tế trong trường hợp xấu nhất là $T(m)$, thì **chi phí khấu hao** trung bình cho mỗi thao tác là:

$$\hat{c} = \frac{T(m)}{m}$$

Lưu ý cốt lõi: Phân tích khấu hao vẫn là một sự bảo đảm trong **trường hợp tồi nhất** (worst-case guarantee) cho toàn bộ chuỗi thao tác. Nó hoàn toàn **không dựa trên giả định xác suất** của dữ liệu đầu vào.

5.2 Ba Phương pháp Phân tích Khấu hao

- Phương pháp Cộng dồn (Aggregate Method):** Tính tổng chi phí trường hợp xấu nhất $T(m)$ của toàn bộ chuỗi m thao tác rồi chia đều cho m : $\hat{c} = T(m)/m$.
- Phương pháp Kế toán (Accounting Method / Banker's Method):** Gán một mức phí giả định $\hat{c}_i$ cho mỗi thao tác. Thao tác trả tiền để tích lũy thành “tín dụng” (credit). Thao tác đắt tiền sẽ rút tín dụng đã tích lũy để bù đắp chi phí thực tế. **Bất biến bắt buộc:** Tổng số dư tín dụng không bao giờ được âm tại bất kỳ thời điểm nào.
- Phương pháp Thế năng (Potential Method / Physicist's Method):** Gán một hàm thế năng $\Phi(D)$ cho trạng thái dữ liệu D thỏa mãn $\Phi(D_0) = 0$ và $\Phi(D_i) \geq 0$. Chi phí khấu hao của thao tác thứ i được định nghĩa:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

Khi cộng dồn qua m thao tác, các đại lượng thế năng bị triệt tiêu theo chuỗi lồng nhau (telescoping):

$$\sum_{i=1}^m \hat{c}_i = \sum_{i=1}^m c_i + \Phi(D_m) - \Phi(D_0) \geq \sum_{i=1}^m c_i$$

5.3 Nghiên cứu Điển hình: Cơ chế Gấp đôi Dung lượng của Mảng Động

Xét lớp ‘DynamicArray’ (tương tự như ‘list’ trong Python hay ‘std::vector’ trong C++):

- Khi chèn vào mảng chưa đầy, chi phí là 1 thao tác ghi nhớ.
- Khi mảng đầy, ta cấp phát mảng mới có kích thước gấp đôi và sao chép toàn bộ phần tử cũ sang mảng mới.
- Với n phần tử, việc sao chép diễn ra tại $n = 1, 2, 4, 8, 16, \dots, 2^k$.
- Tổng số lần sao chép phần tử qua mọi lần mở rộng:

$$1 + 2 + 4 + 8 + \dots + 2^k = 2^{k+1} - 1 < 2n$$

- Tổng chi phí cho chuỗi n thao tác append: $T(n) = n$ (thao tác ghi) + $2n$ (sao chép) $\leq 3n$.

- Chi phí khấu hao mỗi thao tác: $\hat{c} = \frac{3n}{n} = 3 = \Theta(1)$.

So sánh Chiến lược Gấp đôi và Mở rộng Cố định

Nếu mỗi lần đầy mảng, ta chỉ tăng thêm một hằng số k phần tử (ví dụ +1000 ô nhớ), thì qua n thao tác mảng sẽ phải sao chép lại $\approx n/k$ lần. Tổng số phép sao chép là $\sum_{i=1}^{n/k} i \cdot k \approx \Theta(n^2)$. Khi đó chi phí khấu hao cho mỗi thao tác chèn tăng vọt lên $\Theta(n)$! Chiến lược mở rộng theo cấp số nhân (gấp đôi) là điều kiện bắt buộc để đạt được chi phí khấu hao $O(1)$.

6. Bài tập Thực hành & Lời giải Chi tiết

6.1 Bài toán 1: Phân cấp Tốc độ Tăng trưởng Tiệm cận

Đề bài: Sắp xếp các hàm sau theo thứ tự tốc độ tăng trưởng từ chậm nhất đến nhanh nhất:

$$f_1(n) = n^{1.5}, \quad f_2(n) = 1000n, \quad f_3(n) = n \log_2 n, \quad f_4(n) = 2^{\sqrt{n}}, \quad f_5(n) = n!, \quad f_6(n) = 2^n, \quad f_7(n) = n^2$$

Lời giải Bài toán 1

Khảo sát số mũ và lôgarit:

1. $f_2(n) = 1000n = \Theta(n)$ (tuyến tính)
2. $f_3(n) = n \log_2 n$ (tuyến tính lôgarit, tăng nhanh hơn n nhưng chậm hơn $n^{1.5}$)
3. $f_1(n) = n^{1.5}$
4. $f_7(n) = n^2$
5. $f_4(n) = 2^{\sqrt{n}}$: Lấy lôgarit cơ số 2: $\log_2(2^{\sqrt{n}}) = \sqrt{n}$, trong khi $\log_2(n^2) = 2 \log_2 n$. Do $\sqrt{n} \succ \log n$, hàm $2^{\sqrt{n}}$ tăng nhanh hơn mọi đa thức nhưng chậm hơn 2^n (vì $\sqrt{n} < n$).
6. $f_6(n) = 2^n$ (hàm mũ)
7. $f_5(n) = n! \approx \sqrt{2\pi n}(n/e)^n$ (giai thừa, lớn hơn hàm mũ 2^n).

Thứ tự chuẩn xác:

$$1000n \prec n \log n \prec n^{1.5} \prec n^2 \prec 2^{\sqrt{n}} \prec 2^n \prec n!$$

6.2 Bài toán 2: Phân tích Vòng lặp Lồng nhau

Đề bài: Xác định độ phức tạp thời gian tiệm cận chặt Θ của đoạn code sau:

```
1 def compute(n):
2     total = 0
3     i = 1
4     while i <= n:
5         j = 1
6         while j <= n:
7             total += 1
8             j *= 2
9         i += 1
10    return total
```

Lời giải Bài toán 2

- Vòng lặp ngoài khởi tạo $i = 1$ và tăng $i \leftarrow i + 1$ cho đến khi $i > n$. Vòng lặp này chạy đúng n lần.
- Với mỗi giá trị của i , vòng lặp trong khởi tạo $j = 1$ và nhân đôi $j \leftarrow j \times 2$. Chuỗi giá trị của j là $1, 2, 4, 8, \dots, 2^k$. Vòng lặp dừng khi $2^k > n \implies k = \lfloor \log_2 n \rfloor + 1$. Vòng trong thực hiện $\Theta(\log n)$ thao tác hoàn toàn độc lập với i .
- Theo Quy tắc Nhân: $T(n) = \sum_{i=1}^n \Theta(\log n) = n \cdot \Theta(\log n) = \Theta(n \log n)$.

6.3 Bài toán 3: Ứng dụng Định lý Master

Đề bài: Giải các hệ thức truy hồi sau:

1. $T_a(n) = 8T_a(n/2) + n^2$
2. $T_b(n) = 2T_b(n/2) + n \log^2 n$
3. $T_c(n) = 3T_c(n/4) + n \log n$

Lời giải Bài toán 3

Câu 1: Với $T_a(n) = 8T_a(n/2) + n^2$:

- Các hệ số: $a = 8, b = 2, f(n) = n^2$.
- Tính $n^{\log_b a} = n^{\log_2 8} = n^3$.
- So sánh $f(n) = n^2$ với n^3 : Ta có $n^2 = O(n^{3-\epsilon})$ với $\epsilon = 1 > 0$. Đây là **Trường hợp 1**.
- **Kết luận:** $T_a(n) = \Theta(n^3)$.

Câu 2: Với $T_b(n) = 2T_b(n/2) + n \log^2 n$:

- $a = 2, b = 2, f(n) = n \log^2 n \implies k = 1, p = 2$.
- Do $a = b^k$ ($2 = 2^1$) và $p = 2 > -1$, đây là **Trường hợp 2 Mở rộng**.
- **Kết luận:** $T_b(n) = \Theta(n^k \log^{p+1} n) = \Theta(n \log^3 n)$.

Câu 3: Với $T_c(n) = 3T_c(n/4) + n \log n$:

- $a = 3, b = 4, f(n) = n \log n$.
- $n^{\log_b a} = n^{\log_4 3} \approx n^{0.7925}$.
- $f(n) = n \log n = \Omega(n^{0.7925+\epsilon})$ với $\epsilon \approx 0.2 > 0$.
- Kiểm tra điều kiện chính quy: $a \cdot f(n/b) = 3(n/4 \log(n/4)) \leq \frac{3}{4}n \log n$. Chọn $c = 0.8 < 1$, điều kiện thỏa mãn.
- Đây là **Trường hợp 3**.
- **Kết luận:** $T_c(n) = \Theta(n \log n)$.

7. Bảng Tra cứu Công thức & Tóm tắt Nhanh

7.1 Các Công thức Tổng Cốt lõi

- **Tổng cấp số cộng:** $\sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$
- **Tổng bình phương:** $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$
- **Cấp số nhân hữu hạn ($r \neq 1$):** $\sum_{i=0}^k r^i = \frac{r^{k+1}-1}{r-1}$
- **Cấp số nhân lùi vô hạn ($|r| < 1$):** $\sum_{i=0}^\infty r^i = \frac{1}{1-r}$
- **Chuỗi điều hòa:** $\sum_{i=1}^n \frac{1}{i} = \ln n + \gamma + O(1/n) = \Theta(\log n)$
- **Xấp xỉ Stirling cho giai thừa:** $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \implies \log(n!) = \Theta(n \log n)$

7.2 Bảng Quyết định Định lý Master

Cho hệ thức $T(n) = aT(n/b) + \Theta(n^k \log^p n)$:

So sánh a và b^k	Số mũ p	Thành phần Chi phối	Nghiệm $T(n)$
$a > b^k$	Mọi p	Tầng lá (Chiều sâu $\log_b n$)	$\Theta(n^{\log_b a})$
$a = b^k$	$p > -1$	Mọi tầng đều nhau	$\Theta(n^k \log^{p+1} n)$
$a = b^k$	$p = -1$	Trường hợp biên	$\Theta(n^k \log \log n)$
$a = b^k$	$p < -1$	Hội tụ	$\Theta(n^k)$
$a < b^k$	$p \geq 0$	Chi phí tại gốc	$\Theta(n^k \log^p n)$
$a < b^k$	$p < 0$	Chi phí tại gốc	$O(n^k)$